CUNEF UNIVERSIDAD

Relational databases · Databases practice

Normalization: summary

1. **Why normalize**: Redundancy and anomalies
2. **Dependencies**: Functional dependency · Keys and attribute closure · Full and partial dependency · Transitive dependency · Multivalued dependency · Join dependency
3. **Normal forms**: First normal form (1NF) · Second normal form (2NF) · Third normal form (3NF) · Boyce-Codd normal form (BCNF) · Fourth normal form (4NF) · Fifth normal form (5NF)
4. **Decomposition**: Lossless-join decomposition · Dependency preservation

Why normalize

Redundancy and anomalies

A table that mixes several facts repeats data. Every repetition invites three anomalies: update, insertion and deletion.

- **Update anomaly**: rename course C10 in one row only, and the same course now has two titles.
- **Insertion anomaly**: a new course with no students yet cannot be stored, because course id alone is not a key and student id cannot be NULL.
- **Deletion anomaly**: delete the last enrolment of a course, and the course's title and credits disappear with it.

Enrolment: one wide table (bold = primary key)

student id	**course id**	student name	course title	credits	dept	grade
S01	C10	Ana Ruiz	Databases	6	Computing	8.5
S01	C20	Ana Ruiz	Statistics	6	Maths	7.0
S02	C10	Luis Gil	Databases	6	Computing	6.5
S03	C10	Eva Sanz	Databases	6	Computing	5.0

> **Example.** Normalization splits this table so each fact is stored once: students in one table, courses in another, departments in a third, and enrolments (student, course, grade) linking them.

> **Common mistake.** Thinking redundancy only wastes space. The real cost is inconsistency: two rows that should agree can come to disagree.

Dependencies

Functional dependency

`X → Y` **(X determines Y) means: two rows that agree on X always agree on Y. Knowing X, there is only one possible Y.**

- Read `X → Y` as "for each X, a single Y".
- Trivial: `X → Y` when Y is part of X (it always holds).
- Every key determines every attribute of its table.

> **Example.** In the enrolment table: `student id → student name`, `course id → course title, credits, dept`, and `student id, course id → grade`.

> **Common mistake.** Reading the arrow backwards. `student id → student name` does not mean the name determines the id: two students can share a name.

Keys and attribute closure

A candidate key is a minimal set of attributes that determines all the others. The closure X^+ (everything X determines) is how you test it.

- {student id}$^+$ = {student id, student name}: not a key.
- {course id}$^+$ = {course id, course title, credits, dept}: not a key.
- {student id, course id}$^+$ = every attribute, and neither part alone is enough, so it is a **candidate key**.

> **Example.** With `student id → name`, `course id → title, credits, dept` and `student id, course id → grade`, the only candidate key of Enrolment is (student id, course id).

> **Common mistake.** Calling any set that identifies rows a key. (student id, course id, grade) also identifies rows, but it is a superkey, not a candidate key, because grade is unnecessary.

Full and partial dependency

An attribute depends fully on a composite key when it needs the whole key. It depends partially when part of the key is enough.

- Full: `student id, course id → grade`.
- Partial: `student id → student name`, `course id → course title, credits`.

Which part of the key each attribute needs

Attribute	Depends on	Kind
grade	student id + course id	Full
student name	student id	Partial
course title, credits, dept	course id	Partial

> **Common mistake.** Looking for partial dependencies in a table with a single-attribute key. There are none, so such a table in 1NF is already in 2NF.

Transitive dependency

A transitive dependency is a chain: the key determines A, and A, which is not a key, determines B. B depends on the key only through A.

- Pattern: `key → A → B`, where A is **not** a key and B is not part of a key.
- Fix: move A and B to their own table, with A as its key, and keep A in the original table as a foreign key.

> **Example.** `course id → dept id` and `dept id → dept name`, so `course id → dept name` is transitive.

> **Common mistake.** Calling every chain transitive. If A is a candidate key, `key → A → B` causes no redundancy.

Multivalued dependency

`X →→ Y` **means: X determines a set of Y values, independent of the rest of the row. It appears when one table holds two independent lists about X.**

Course offering: every teacher × every book (bold = key)

course	teacher	book
Databases	Ruiz	Elmasri
Databases	Ruiz	Date
Databases	Gil	Elmasri
Databases	Gil	Date

> **Example.** Adding a third book to Databases needs two new rows, one per teacher. That is the redundancy 4NF removes.

> **Common mistake.** Seeing a multivalued dependency whenever an attribute has several values. It needs **two** lists that are independent of each other.

Join dependency

A table has a join dependency when it can be rebuilt exactly by joining some of its projections. Multivalued dependencies are the two-part case.

- It only holds if the business rule **always** holds, never just because of the current data.

- It is removed by 5NF, which stores the pairs instead of the triple.

> **Common mistake.** Decomposing a ternary table into pairs without such a rule. Without the rule, the join of the pairs invents combinations and the decomposition is not lossless.

Normal forms

First normal form (1NF)

One cell, one value. Every attribute is atomic, there are no repeating groups, and the table has a primary key.

Not in 1NF: a list in one cell

student id	student name	courses
S01	Ana Ruiz	C10, C20
S02	Luis Gil	C10, C11

In 1NF: one course per row, key (student id, course id)

student id	course id	student name
S01	C10	Ana Ruiz
S01	C20	Ana Ruiz
S02	C10	Luis Gil
S02	C11	Luis Gil

> **Common mistake.** Thinking 1NF is the end. The 1NF table above repeats each name per course: it now has a partial dependency, which 2NF removes.

Second normal form (2NF)

The whole key. In 1NF, and every non-key attribute depends on the entire key, with no partial dependencies.

Enrolment after 2NF (bold = key)

Table	Columns
Student	**student id**, student name
Course	**course id**, course title, credits, dept id, dept name
Enrolment	**student id**, **course id**, grade

How to reach 2NF

1. Look at the primary key. If it has a single attribute, 2NF already holds.
2. If it is composite, ask about each non-key attribute: is part of the key enough to know it?
3. Move the ones that depend on part of the key to a new table whose key is that part.
4. In the source table keep the full key and only what depends on all of it.

> **Example.** Ana Ruiz's name is now stored once, and a course can exist without enrolments.

> **Common mistake.** Forgetting to keep the key parts in the original table. Enrolment must keep student id and course id: they are the foreign keys that link the pieces back.

Third normal form (3NF)

Nothing but the key. In 2NF, and no non-key attribute depends on another non-key attribute: there are no transitive dependencies.

After 3NF (bold = key)

Table	Columns
Student	**student id**, student name
Department	**dept id**, dept name

Table	Columns
Course	**course id**, course title, credits, dept id → Department
Enrolment	**student id**, **course id**, grade

How to reach 3NF

1. Look for chains: the key determines A and A determines B. Then B depends on A, not on the key.
2. Move A and B to a new table whose key is A.
3. Keep A in the source table too: it is the reference (foreign key) that links them.
4. Repeat while chains remain: each link ends up with its own table.

Example. "Every non-key attribute depends on the key, the whole key, and nothing but the key."

Common mistake. Removing dept id from Course as well. Then nothing says which department a course belongs to: the decomposition loses information.

Boyce-Codd normal form (BCNF)

Every determinant is a key. For every non-trivial $X \to Y$, X must be a superkey. There are no exceptions for prime attributes.

In 3NF but not BCNF: tutor → subject (bold = key)

student	subject	tutor
Ana	Maths	Pérez
Luis	Maths	Pérez
Ana	Physics	Gómez

How to reach BCNF

1. Write down the left-hand side of every functional dependency.
2. For each one ask: does it identify each row of the table by itself? If not, it is a problem, even if what it determines is part of a key (3NF tolerates it, BCNF does not).
3. Move the determinant and what it determines to a new table, with the determinant as its key.
4. In the source table keep the determinant (as a reference) and remove what it determines.
5. Watch out: some dependency may stop being checkable within a single table. Sometimes that is unavoidable.

Common mistake. Assuming every 3NF table is in BCNF. They differ only when there are overlapping candidate keys, but then the difference is real.

Fourth normal form (4NF)

One fact per table. In BCNF, and for every non-trivial multivalued dependency $X \twoheadrightarrow Y$, X is a superkey.

After 4NF (bold = key)

Table	Columns
CourseTeacher	**course**, **teacher**
CourseBook	**course**, **book**

How to reach 4NF

1. Look for tables where almost everything is key: the same X with several values of Y and several values of Z.
2. Ask: for the same X, are the values of Y and Z combined every way, with no relationship between them? If so, there is a multivalued dependency $X \twoheadrightarrow Y \mid Z$.

3. Store each fact in its own table: one with X and Y, another with X and Z. The key of each is its two attributes.
4. Check that nothing is lost: joining the two tables on X must give exactly the original table.

Example. Adding a third book to Databases is now one row in CourseBook, whatever the number of teachers.

Common mistake. Splitting a triple that is not independent. If each teacher uses their own books, (course, teacher, book) is a single fact and is already in 4NF.

Fifth normal form (5NF)

Nothing that can be rebuilt from its parts. In 4NF, and every join dependency is implied by the candidate keys.

After 5NF (bold = key)

Table	Columns
SupplierPart	**supplier**, **part**
PartProject	**part**, **project**
SupplierProject	**supplier**, **project**

How to reach 5NF

1. Look for tables with three or more attributes that are all key and have no multivalued dependencies.
2. Ask: is there a rule of the form "if A–B, B–C and A–C all hold, then A–B–C holds"? Then the table can be rebuilt from its pairs.
3. Replace it with one table per pair: (A, B), (B, C) and (A, C).
4. Check that with only two of the three pairs, rows that never existed appear: all of them are needed.
5. Careful: this is only correct if the rule always holds. If it is a coincidence of the current data, the table with all three attributes is the right one.

Common mistake. Applying 5NF because the current rows happen to decompose. Without a business rule that guarantees it, the next row inserted breaks the join.

Decomposition

Lossless-join decomposition

A decomposition is lossless when joining the pieces gives back exactly the original table: no rows lost and no spurious rows.

- Lossless ⇔ (R1 ∩ R2) → R1 or (R1 ∩ R2) → R2.
- Spurious rows mean information is lost: you can no longer tell which rows were real.

Common mistake. Checking only that every attribute appears in some piece. Having every column is not enough; the pieces must join back correctly.

Dependency preservation

A decomposition preserves dependencies when every dependency can still be checked inside a single table, without joins.

What each target guarantees

Target	Lossless	Preserves dependencies	Redundancy from FDs
3NF	Always	Always possible	Some may remain
BCNF	Always	Not always	None

> **Common mistake.** Believing higher is always better. Each form trades checks that are easy to enforce against redundancy. Choose with the business rules in mind.